DISEÑO EN HARDWARE DEL ALGORITMO HAMMING QUACI-CYLIC (HQC) PARA ENCRIPTACIÓN POSTCUÁNTICA

Camilo Home Buriticá



Escuela de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería
Santiago de Cali
Diciembre 2025

DISEÑO EN HARDWARE DEL ALGORITMO HAMMING QUACI-CYLIC (HQC) PARA ENCRIPTACIÓN POSTCUÁNTICA

Camilo Home Buriticá

Documento presentado a
Programa Académico de Ingeniería Electrónica
como propuesta de trabajo de grado

Director: Jaime Velasco Medina, Ph.D., M.Sc., Ing.
Grupo de Investigación en Bionanoelectrónica (BIONANO)

Codirectora: Claudia Patricia Rentería Mejía, Ph.D., M.Sc., Ing.
Grupo de Investigación en Bionanoelectrónica (BIONANO)



Escuela de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería
Santiago de Cali
Diciembre 2025

TABLA DE CONTENIDO

Pag.

RESUMEN **1**

1. INTRODUCCIÓN **2**

1.1. CONTEXTUALIZACIÓN 2
1.2. PLANTEAMIENTO DEL PROBLEMA 4
1.3. OBJETIVOS 6
1.3.1. Objetivo General 6
1.3.2. Objetivos Específicos 6
1.4. CONTRIBUCIÓN DE ESTE TRABAJO 6
1.5. ORGANIZACIÓN DE LOS CAPÍTULOS 6

2. ANTECEDENTES **7**

2.1. ANTECEDENTES NACIONALES 8
2.2. ANTECEDENTES INTERNACIONALES 8
2.3. DISCUSIÓN 8

3. FUNDAMENTACIÓN TEÓRICA **8**

3.1. CRIPTOGRAFÍA PRE-CUÁNTICA 8
3.1.1. Criptografía simétrica 9
3.1.2. Criptografía de clave publica 9
3.1.3. Relación con la criptografía post-cuántica 10
3.2. CRIPTOGRAFÍA POST-CUÁNTICA 10
3.2.1. Criptografía basada en retículos (*lattice-based cryptography*) 10
3.2.2. Criptografía basada en funciones hash 11
3.2.3. Criptografía basada en codigos correctores de errores 11
3.2.4. Limitaciones de la criptografía clásica frente a computación cuántica 12
3.2.5. Clasificación general de esquemas post-cuánticos 13
3.2.5.1. Basados en lattices 13
3.2.5.2. Basados en hash 13
3.2.5.3. Basados en códigos 13
3.2.6. Rol de los esquemas KEM en protocolos modernos (TLS, IoT) 13
3.3. Criptografía basada en códigos 13
3.3.1. Códigos correctores de errores 13
3.4. Algoritmo HQC (Hamming Quasi-Cyclic) 13
3.5. Representación matemática y estructuras de datos 13
3.6. Operaciones criptográficas involucradas 13

4. IMPLEMENTACIÓN FUNCIONAL DE KEM HQC **14**

4.1. INTRODUCCIÓN 14
4.2. Fase 2: Estructuración del algoritmo HQC en alto nivel utilizando Python 14
4.2.1. Estructura y descripción de funciones 15
4.3. Fase 3: Descomposición funcional del algoritmo HQC 18
4.3.1. Generación de claves (Keygen) 19
4.3.1.1. Expansión de semillas 19
4.4. CONCLUSIONES 20

5. PRUEBAS Y RESULTADOS 21

5.1. INTRODUCCIÓN 21
5.2. PRUEBA N 21
5.2.1. Descripción 21
5.2.2. Procedimiento 21
5.2.3. Análisis 21

6. CONCLUSIONES GENERALES 22

7. TRABAJOS FUTUROS 22

RESUMEN

La criptografía post-cuántica surge como un campo esencial para asegurar la privacidad, integridad y confiabilidad de la información frente al avance de la computación cuántica, la cual tiene el potencial de comprometer los sistemas de seguridad actuales que protegen las comunicaciones digitales. Dentro de este contexto, el presente trabajo se enfoca en el algoritmo HQC (Hamming Quasi-Cyclic), un mecanismo para el intercambio seguro de claves que ha sido seleccionado por el Instituto Nacional de Estándares y Tecnología (NIST) como parte de su proceso de estandarización de nuevos algoritmos resistentes a este tipo de amenazas. El proyecto aborda específicamente el diseño e implementación en hardware del algoritmo, evaluando su viabilidad y optimizando su rendimiento en plataformas reprogramables (FPGA), con el fin de contribuir al desarrollo de soluciones prácticas que combinen seguridad, velocidad y eficiencia energética.

El propósito general consiste en desarrollar una implementación hardware del algoritmo HQC conforme a las especificaciones del NIST, buscando optimizar su velocidad de procesamiento, su utilización de recursos lógicos y su consumo de potencia. Para lograrlo, la investigación se estructura en varias etapas. En primer lugar, se implementa una versión funcional del algoritmo en alto nivel para lograr una comprensión clara de sus operaciones fundamentales y sentar un modelo de referencia. A partir de este modelo, se procede al diseño detallado de los bloques funcionales que componen el algoritmo. Estos diseños se traducen posteriormente en una arquitectura digital pura mediante lenguaje para descripción de hardware, donde se toman decisiones orientadas a lograr un equilibrio entre el rendimiento, la cantidad de recursos lógicos y el consumo energético. Finalmente, se lleva a cabo una evaluación comparativa del desempeño del diseño propuesto frente a otras implementaciones hardware documentadas, analizando métricas concretas como los tiempos de ejecución, la utilización de recursos del FPGA y la potencia consumida.

De este modo, el trabajo no solo valida la viabilidad práctica del algoritmo HQC en hardware, sino que también sienta las bases técnicas para su futura integración en sistemas embebidos y redes seguras, aportando una pieza fundamental en la construcción de infraestructuras de ciberseguridad resistentes a la computación cuántica y la futura migración de la seguridad de la información en la era post-cuántica.

Palabras Clave: Criptografía postcuántica, computación cuántica, diseño en FPGA, encapsulación de clave, ciberseguridad.

1. INTRODUCCIÓN

1.1 CONTEXTUALIZACIÓN

El desarrollo de la computación cuántica está avanzando rápidamente y se ha convertido en uno de los mayores desafíos para los métodos de criptografía convencionales, comprometiendo potencialmente la seguridad de la información a escala global. A diferencia de la computación clásica, los computadores cuánticos explotan fenómenos propios de la mecánica cuántica, como la superposición y el entrelazamiento, permitiendo la ejecución de algoritmos con capacidades computacionales muy superiores para ciertos tipos de problemas. En particular, estos avances impactan directamente los supuestos de complejidad matemática sobre los cuales se fundamentan los principales algoritmos criptográficos actuales, tales como RSA y los esquemas basados en criptografía de curva elíptica (ECC). El algoritmo de Shor, por ejemplo, permite la factorización de enteros grandes y la resolución del problema del logaritmo discreto en tiempo polinomial, anulando directamente la seguridad de estos algoritmos cuando se dispone de un computador cuántico suficientemente potente [1]. Como consecuencia, una gran parte de la infraestructura de seguridad digital actual se ve amenazada ante la eventual madurez de esta tecnología.

Este escenario ha motivado a la comunidad científica internacional y, en particular, al Instituto Nacional de Estándares y Tecnología (NIST) de los Estados Unidos, a liderar un proceso de estandarización de nuevos algoritmos criptográficos capaces de resistir ataques tanto clásicos como cuánticos. Dicho esfuerzo ha dado origen al campo emergente de la criptografía post-cuántica, cuyo objetivo es desarrollar esquemas criptográficos basados en problemas matemáticos que se consideran intratables incluso para computadores cuánticos. Este proceso de estandarización ha evaluado múltiples familias de algoritmos, incluyendo aquellos basados en retículos, códigos correctores de errores y funciones hash, buscando alternativas seguras y prácticas para su despliegue a gran escala.

Dentro de este contexto, el algoritmo Hamming Quasi-Cyclic (HQC) ha sido seleccionado por el NIST en su cuarta ronda de evaluación como el quinto sistema criptográfico en ser estandarizado, específicamente como un mecanismo de encapsulación de claves (KEM), en reemplazo de ML-KEM [2]. HQC pertenece a la familia de esquemas criptográficos basados en códigos de corrección de errores y fundamenta su seguridad en la dificultad del problema de decodificación de síndromes en códigos quasi-cíclicos, un problema que no cuenta con algoritmos cuánticos eficientes conocidos. A diferencia de los algoritmos basados en retículos (*lattices*) que dominan el proceso de estandarización, HQC ofrece una estructura alternativa que diversifica el panorama criptográfico post-cuántico, reduciendo el riesgo asociado a depender de una sola familia de problemas mate-

máticos. Además, su diseño presenta ventajas en términos de simplicidad conceptual y robustez frente a ciertos tipos de ataques estructurales.

Sin embargo, la adopción real de HQC en sistemas de transmisión de información y aplicaciones prácticas dependerá en gran medida de la viabilidad de su implementación eficiente en hardware. Muchos de los escenarios donde se requiere criptografía post-cuántica, como sistemas embebidos, dispositivos IoT, infraestructura de red y sistemas de comunicaciones seguras, presentan restricciones estrictas en cuanto a recursos, consumo energético y latencia. En este sentido, evaluar el desempeño de HQC en plataformas hardware es un paso fundamental para validar su aplicabilidad más allá del ámbito teórico o del software de propósito general.

En este marco, el presente proyecto tiene como objetivo desarrollar una arquitectura hardware para el algoritmo HQC, siguiendo las especificaciones más recientes publicadas por el NIST. El diseño se enfocará en optimizar el rendimiento en términos de velocidad de procesamiento, utilización de recursos lógicos y eficiencia energética, aspectos críticos en implementaciones criptográficas modernas. Para ello, se emplearán descripciones a nivel de transferencia de registros (RTL) en VHDL, herramientas de síntesis y análisis como Quartus Prime, y una plataforma FPGA Cyclone IV (DE2–115) para la validación funcional y la evaluación de desempeño. Esta aproximación permitirá analizar de manera detallada los compromisos entre seguridad, rendimiento y costo de implementación.

De manera complementaria, como parte del proceso de diseño, se abordará inevitablemente y de forma sistemática la implementación de las operaciones criptográficas y las representaciones matemáticas involucradas en el funcionamiento interno del algoritmo HQC. Este enfoque permitirá que el diseño propuesto sirva no solo como una implementación funcional, sino también como una guía de referencia para comprender la traducción de los conceptos teóricos del algoritmo a estructuras hardware concretas.

Este trabajo no solo busca validar la correcta implementación funcional del algoritmo HQC en hardware, sino también contribuir a la investigación y desarrollo de soluciones criptográficas capaces de enfrentar las amenazas emergentes de la era cuántica. Al explorar arquitecturas eficientes y reproducibles en FPGA, se aporta evidencia práctica sobre la viabilidad de la criptografía post-cuántica en escenarios reales y así aportar al fortalecimiento del camino hacia una transición segura de las infraestructuras criptográficas actuales.

1.2 PLANTEAMIENTO DEL PROBLEMA

El avance de las tecnologías de la información y la comunicación (TIC) ha transformado completamente la manera en la que los seres humanos nos comunicamos e intercambiamos información. Desde la unificación de las telecomunicaciones y las redes informáticas hasta el almacenamiento de datos y la implementación del internet de las cosas (IoT), la transmisión de información se ha convertido en una actividad fundamental en nuestro mundo moderno. A diario se transmiten grandes cantidades de datos, de los cuales muchos son de carácter sensible o privado. Con el fin de garantizar la confidencialidad, integridad y autenticidad de la información se utiliza la criptografía, la cual se encarga de desarrollar métodos de cifrado para modificar la representación de ciertos mensajes haciéndolos incomprensibles a receptores que no tengan la autorización para acceder a su contenido [3].

Los algoritmos criptográficos basados en problemas matemáticos complejos, como RSA (uso de clave publica), AES (cifrado simétrico) y ECC (criptografía de curva elíptica), han brindado un alto nivel de seguridad durante décadas. Sin embargo, el acelerado desarrollo de la computación cuántica a lo largo de los últimos años ha demostrado que algoritmos como el de Shor, puede romper RSA y ECC, y el algoritmo de Grover vulnera la seguridad del algoritmo AES a la mitad, limitándolo a ser usado en la era de la computación cuántica solo con una clave de 256-bit [4]. Esto nos deja como resultado un panorama donde la información encriptada bajo estos algoritmos está totalmente expuesta, lo que pondría en riesgo la seguridad de aplicaciones bancarias, redes gubernamentales, infraestructura crítica, etc.

Adicionalmente, otro de los principales riesgos emergentes es la práctica conocida como Harvest Now, Decrypt Later (HNDL), que en español significa "recolectar ahora, desencriptar después", la cual es una estrategia que se basa en la adquisición y almacenamiento a largo plazo de información actualmente cifrada con el objetivo de descifrarla en un futuro cuando se disponga de la suficiente potencia de cómputo cuántico [5]. Se estima que hay un alto número de actores maliciosos, especialmente atacantes de entes gubernamentales, que estarían robando datos cifrados muy sensibles con fines políticos y sociales.

La aparición de este tipo de estrategias ha impulsado a la comunidad criptográfica a desarrollar con urgencia nuevos algoritmos de encriptación, lo cual ha dado origen a un nuevo estándar que reemplazará la criptografía de clave pública actual con la criptografía post-cuántica, termino con el que ha sido denominada la criptografía resistente a los ataques cuánticos. Este Proceso de estandarización se ha llevado a cabo por el Instituto Nacional de Estándares y Tecnología (NIST) con el fin de seleccionar los algoritmos que reemplacen los algoritmos tradicionales de seguridad para firma digital y cifrado e intercambio de claves [6].

Dentro de estos esfuerzos, el algoritmo Hamming Quasi-Cyclic (HQC) fue seleccionado en la categoría Key Encapsulation Mechanism (KEM) como el quinto y último algoritmo en ser estandarizado hasta la fecha. En agosto de 2025 se publicó su última especificación, la cual introduce optimizaciones en el mecanismo de encapsulación/decapsulación, corrigiendo detalles criptográficos que podían introducir debilidades prácticas [7]. Este algoritmo servirá como respaldo de ML-KEM, uno de los algoritmos actuales para criptografía de clave pública dado que el HQC se basa en una matemática diferente a la de ML-KEM, lo cual podría ser importante en caso de que se descubriesen nuevas vulnerabilidades en dicho algoritmo. NIST planea publicar un borrador de estándar que incorpore el algoritmo HQC en el próximo año, y se espera que el estándar final esté listo para 2027 [8].

Dado que la adopción de HQC en sistemas reales dependerá en gran medida de su viabilidad en entornos de hardware se plantea la siguiente pregunta problematizadora:

¿Cómo desarrollar una implementación eficiente en hardware del algoritmo HQC que optimice el rendimiento en términos de velocidad, uso de recursos y consumo de potencia, superando las limitaciones de implementaciones previas basadas en síntesis de alto nivel (HLS)?

Para responder esta pregunta, este trabajo se centrará en la implementación en hardware de HQC, obedeciendo a las especificaciones más recientes del NIST [7], con el objetivo de analizar su desempeño y explorar posibles optimizaciones para su implementación en sistemas embebidos y redes de transmisión de datos de alta seguridad.

1.3 OBJETIVOS

1.3.1 Objetivo General

Desarrollar una implementación en hardware del algoritmo de encriptación postcuántica HQC, siguiendo las especificaciones de NIST, buscando optimizar su rendimiento en velocidad, uso de recursos y consumo de potencia.

1.3.2 Objetivos Específicos

1. Implementar una versión funcional del algoritmo HQC en alto nivel utilizando Python, que facilite comprensión de las principales operaciones.
2. Diseñar los bloques funcionales del algoritmo de encriptación HQC.
3. Desarrollar la arquitectura hardware del algoritmo HQC utilizando VHDL, teniendo en cuenta consumo de recursos, velocidad, y consumo de potencia.
4. Realizar una evaluación comparativa del desempeño con otras implementaciones hardware reportadas, verificando tiempos de ejecución, consumo de recursos y consumo de potencia

1.4 CONTRIBUCIÓN DE ESTE TRABAJO

1.5 ORGANIZACIÓN DE LOS CAPÍTULOS

2. ANTECEDENTES

En este capitulo se presentan los principales trabajos relacionados con la implementación del algoritmo de encapsulación de claves Hamming Quasi-Cyclic (HQC), enfocados en el desarrollo de arquitecturas hardware eficientes que permitan su ejecución en plataformas como sistemas embebidos, FPGAs y otros entornos de computación segura. Estos estudios abordan diferentes estrategias de diseño con el objetivo de mejorar el rendimiento, reducir el uso de recursos y optimizar el consumo energético.

Dentro del material revisado en relación con el presente trabajo, se identificaron cuatro documentos relevantes que proponen distintas arquitecturas hardware para la implementación de HQC. La revisión bibliográfica se concentró en trabajos publicados a partir del año 2020, considerando tanto el carácter reciente del proceso de estandarización liderado por el NIST como la evolución acelerada de las metodologías de diseño a nivel de transferencia de registros (RTL) aplicadas a criptografía post-cuántica.

Cabe resaltar que, hasta la fecha, no se han identificado antecedentes documentados en el ámbito nacional que aborden la implementación del algoritmo criptográfico post-cuántico HQC en hardware reconfigurable o en procesadores dedicados. Esta ausencia de trabajos previos pone de manifiesto una oportunidad significativa para contribuir al desarrollo local en el área de ciberseguridad post-cuántica, particularmente en lo referente a la validación y análisis de implementaciones hardware orientadas a aplicaciones reales.

2.1 ANTECEDENTES NACIONALES

2.2 ANTECEDENTES INTERNACIONALES

2.3 DISCUSIÓN

3. FUNDAMENTACIÓN TEÓRICA

3.1 CRIPTOGRAFÍA PRE-CUÁNTICA

La criptografía pre-cuántica hace referencia al conjunto de algoritmos y protocolos criptográficos diseñados bajo el supuesto de un modelo de computación clásico, es decir, cuando se asume que el adversario dispone únicamente recursos computacionales tradicionales. Estos esquemas constituyen la base de la seguridad en la comunicación digital actual y han sido ampliamente utilizados durante décadas en todo lo referente a protección e intercambio de datos.

El diseño de la criptografía pre-cuántica se fundamenta en la dificultad computacional de ciertos problemas matemáticos que, hasta el momento, no cuentan con soluciones eficientes en computadores clásicos. Entre estos problemas destacan la factorización de enteros grandes, el problema del logaritmo discreto y sus variantes en diferentes estructuras algebraicas. La seguridad de los principales sistemas criptográficos actuales no se basa en la imposibilidad matemática absoluta de resolver los problemas subyacentes, sino en el hecho de que su resolución resulte computacionalmente inviable bajo los recursos y modelos de cómputo disponibles. En este contexto, un problema se considera seguro cuando el tiempo, la memoria o la energía necesarios para resolverlo exceden de manera práctica cualquier capacidad razonable de un adversario.

Este enfoque permite construir esquemas criptográficos robustos apoyados en supuestos de complejidad bien establecidos, pero también implica que la seguridad está intrínsecamente ligada al modelo computacional asumido. Por esta razón, avances significativos en las capacidades de cómputo, como los introducidos por la computación cuántica, pueden alterar de forma sustancial estos supuestos y comprometer la seguridad de algoritmos que hoy se consideran confiables.

3.1.1 Criptografía simétrica

La criptografía simétrica se basa en el uso de una única clave secreta compartida entre las partes que se comunican, la cual se emplea tanto para el cifrado como para el descifrado de la información. Estos algoritmos se caracterizan por su alta eficiencia computacional y bajo consumo de recursos, lo que los hace especialmente adecuados para el cifrado de grandes volúmenes de datos. Sin embargo, su principal limitación radica en la distribución segura de la clave, ya que ambas partes deben compartirla previamente a través de un canal confiable.

Supongamos que Alice y Bob desean intercambiar mensajes de forma confidencial. Ambos acuerdan previamente una clave secreta, por ejemplo una contraseña compartida. Alice cifra su mensaje utilizando esa clave y lo envía a Bob, al recibirlo, Bob usa la misma clave para descifrar el mensaje y recuperar la información original. En este caso, la seguridad del sistema depende de que la clave compartida se mantenga en secreto, y el principal desafío radica en cómo intercambiar dicha clave sin que se filtre o se intercepte antes de iniciar la comunicación.

Este tipo de criptografía la conforman algoritmos como AES para cifrado y funciones hash como SHA-2 y SHA-3. A diferencia de la criptografía de clave pública, estos algoritmos se basan en operaciones simples a nivel de bits y bloques, y no en problemas algebraicos complejos. Aunque los algoritmos simétricos también se ven afectados por algoritmos cuánticos, como Grover, su impacto es menos severo, ya que puede mitigarse aumentando el tamaño de las claves. Por esta razón, gran parte de la criptografía simétrica actual se considera parcialmente resistente al modelo cuántico.

3.1.2 Criptografía de clave publica

La criptografía de clave publica, también conocida como criptografía asimétrica, utiliza un par de claves matemáticamente relacionadas: una clave pública, utilizada para cifrar la información, y una clave privada para descifrarla. Este enfoque elimina la necesidad de compartir una clave secreta previamente, facilitando la gestión de claves en sistemas distribuidos. Sin embargo, la desventaja es que estos algoritmos presentan un mayor costo computacional, por lo que se suelen emplear para el intercambio seguro de claves o autenticación y no para el cifrado directo de grandes cantidades de datos.

De manera análoga, supongamos ahora que Bob quiere recibir mensajes cifrados de cualquier persona sin tener que compartir una clave secreta con antelación. Para ello, genera un par de claves: una clave pública, que puede distribuir libremente, y una clave privada, que mantiene en secreto. Alice cifra su mensaje utilizando la clave pública de Bob y se lo envía. Solamente Bob,

con su clave privada, puede descifrar el mensaje.

Dentro de la criptografía de clave pública los algoritmos más representativos son RSA, cuya seguridad se basa en la dificultad de factorizar números enteros primos de gran tamaño y ECC, que se fundamenta en la dificultad del logaritmo discreto sobre curvas elípticas. Estos esquemas permiten establecer canales seguros sin necesidad de compartir previamente una clave secreta.

3.1.3 Relación con la criptografía post-cuántica

La criptografía pre-cuántica constituye el punto de partida conceptual y tecnológico para el desarrollo de la criptografía post-cuántica. El análisis de sus debilidades frente al modelo cuántico ha permitido identificar nuevos criterios de diseño, concentrándose en problemas matemáticos que no presentan vulnerabilidad a algoritmos cuánticos eficientes.

En este sentido, la criptografía post-cuántica no reemplaza completamente a la criptografía pre-cuántica de forma inmediata, sino que plantea un proceso de transición progresivo, donde lo mas seguro es que ambos enfoques coexistirán durante un periodo significativo.

3.2 CRIPTOGRAFÍA POST-CUÁNTICA

La criptografía post-cuántica (PQC) se define como el conjunto de algoritmos criptográficos diseñados para ser seguros frente a adversarios con capacidades tanto clásicas como cuánticas y que a su vez, puedan implementarse en plataformas computacionales convencionales.

A diferencia de la criptografía cuántica, que requiere hardware especializado, la criptografía post-cuántica se enfoca en soluciones puramente algorítmicas, compatibles con los sistemas digitales actuales. Los algoritmos post-cuánticos se basan en problemas matemáticos para los cuales no se conocen algoritmos cuánticos eficientes representando una alta complejidad incluso para este tipo de computación.

3.2.1 Criptografía basada en retículos (*lattice-based cryptography*)

La criptografía basada en retículos se fundamenta en la dificultad de resolver ciertos problemas matemáticos en estructuras algebraicas multidimensionales conocidas como retículos. Dicha estructura puede entenderse como un conjunto regular de puntos en un espacio de alta dimensión generado por combinaciones lineales enteras de un conjunto de vectores base.

Los esquemas criptográficos de esta familia suelen apoyarse en problemas como el Shortest Vector Problem (SVP) o el Learning With Errors (LWE) y sus variantes estructuradas. Estos problemas se consideran computacionalmente intratables tanto para computadores clásicos como cuánticos, y hasta la fecha no se conocen algoritmos cuánticos eficientes que los resuelvan en tiempo polinomial. No obstante, sus implementaciones suelen implicar operaciones aritméticas intensivas y un uso considerable de recursos sobre todo en hardware.

3.2.2 Criptografía basada en funciones hash

La criptografía basada en funciones hash utiliza propiedades fundamentales de estas funciones, como la resistencia a colisiones, la preimagen y la segunda preimagen, para construir primitivas criptográficas seguras. A diferencia de otras familias, no se apoya en problemas algebraicos complejos, sino en la dificultad de invertir o encontrar colisiones en funciones hash bien diseñadas.

En el contexto post-cuántico, esta familia resulta especialmente atractiva debido a su simplicidad conceptual y a que los ataques cuánticos conocidos, como el algoritmo de Grover, solo ofrecen una aceleración cuadrática, la cual puede mitigarse aumentando el tamaño de las salidas hash.

Los esquemas basados en hash se utilizan principalmente para firmas digitales, donde ofrecen altos niveles de seguridad y estructuras relativamente sencillas. Sin embargo, suelen presentar desventajas en términos de tamaño de firmas y costes de almacenamiento, lo que puede limitar su uso en sistemas con restricciones estrictas de memoria.

3.2.3 Criptografía basada en codigos correctores de errores

La criptografía basada en códigos correctores de errores fundamenta su seguridad en la dificultad del problema de decodificación de síndromes en códigos lineales, especialmente cuando estos presentan estructuras específicas como códigos quasi-cíclicos. Este problema consiste en recuperar un mensaje original a partir de una palabra codificada afectada por errores, una tarea que se considera computacionalmente inviable para códigos suficientemente grandes. En el contexto post-cuántico, los códigos correctores no cuentan con algoritmos cuánticos eficientes conocidos que comprometan su seguridad.

Desde el punto de vista de implementación, los esquemas basados en códigos suelen involucrar operaciones binarias simples, como XOR, desplazamientos y sumas modulares, lo que los hace particularmente atractivos para implementaciones en hardware. Sin embargo, una de sus principales limitaciones ha sido históricamente el tamaño elevado de las claves, aunque propuestas modernas

han mitigado este problema mediante el uso de estructuras algebraicas más compactas.

3.2.4 Limitaciones de la criptografía clásica frente a computación cuántica

La criptografía clásica ha sido diseñada bajo el supuesto de un modelo de computación convencional, en el cual la seguridad de los algoritmos se sustenta en la complejidad computacional de ciertos problemas matemáticos considerados intratables en la práctica. Sin embargo, la aparición del paradigma de la computación cuántica introduce un cambio significativo en dicho modelo de amenaza, ya que habilita la ejecución de algoritmos que reducen de forma sustancial la complejidad de problemas fundamentales para la criptografía tradicional.

Uno de los principales impactos de la computación cuántica sobre la criptografía clásica proviene del algoritmo de Shor, el cual permite resolver en tiempo polinomial el problema de la factorización de enteros y el cálculo del logaritmo discreto. Como consecuencia, esquemas criptográficos ampliamente utilizados, tales como RSA, ECC y otros sistemas basados en logaritmo discreto, ven comprometida su seguridad frente a la disponibilidad de computadores cuánticos suficientemente potentes. Este resultado representa una ruptura directa de los supuestos de seguridad sobre los que se construyeron estos algoritmos.

Por otro lado, la criptografía simétrica y las funciones hash también se ven afectadas por la computación cuántica, aunque de manera menos drástica. El algoritmo de Grover proporciona una aceleración cuadrática en la búsqueda exhaustiva de claves, reduciendo la complejidad de un ataque de fuerza bruta de $O(2^n)$ a $O(2^{n/2})$. Si bien este impacto no invalida por completo los esquemas simétricos actuales, sí obliga a reconsiderar los tamaños de clave y de salida de las funciones hash para mantener niveles de seguridad adecuados frente a adversarios cuánticos.

En conjunto, estos avances evidencian que gran parte de la criptografía clásica no puede considerarse segura a largo plazo bajo un modelo de amenaza cuántico. En particular, los algoritmos de clave pública basados en factorización y logaritmo discreto resultan especialmente vulnerables, mientras que los esquemas simétricos requieren ajustes en sus parámetros de seguridad. Esta situación ha motivado el desarrollo y la estandarización de nuevos algoritmos criptográficos diseñados específicamente para resistir ataques tanto clásicos como cuánticos, dando origen al campo de la criptografía post-cuántica.

3.2.5 Clasificación general de esquemas post-cuánticos

3.2.5.1 Basados en lattices

3.2.5.2 Basados en hash

3.2.5.3 Basados en códigos

3.2.6 Rol de los esquemas KEM en protocolos modernos (TLS, IoT)

3.3 Criptografía basada en códigos

3.3.1 Códigos correctores de errores

3.4 Algoritmo HQC (Hamming Quasi-Cyclic)

3.5 Representación matemática y estructuras de datos

3.6 Operaciones criptográficas involucradas

4. IMPLEMENTACIÓN FUNCIONAL DE KEM HQC

4.1 INTRODUCCIÓN

Para esta primera actividad del objetivo de análisis criptográfico, se empleará una metodología de investigación documental analítica con enfoque sistemático, estructurada en tres fases consecutivas: FALTA
FALTA
FALTA
FALTA

4.2 Fase 2: Estructuración del algoritmo HQC en alto nivel utilizando Python

En esta sección se presenta la implementación del esquema criptográfico HQC-128 en Python, diseñada para reflejar fielmente el flujo y las operaciones definidas en la especificación oficial. La implementación tiene como objetivo principal facilitar la comprensión de todos los bloques funcionales del algoritmo, incluyendo la generación de claves, el cifrado y descifrado de mensajes, así como los mecanismos de encapsulación y desencapsulación de claves.

El código ha sido estructurado en clases y funciones claramente definidas, siguiendo un enfoque modular que permite una fácil extensión y análisis de cada componente. De igual manera, se incluyen funciones auxiliares para la manipulación de vectores binarios, operaciones en cuerpos finitos $GF(2^8)$, y codificación/decodificación mediante los esquemas Reed-Solomon y Reed-Muller, necesarios para el funcionamiento de HQC.

La implementación incluye funciones de prueba ya programadas, que permiten comparar los resultados obtenidos con los vectores de prueba oficiales disponibles en el repositorio oficial de HQC, asegurando la validez y consistencia de la implementación.

La implementación desarrollada se encuentra alojada en el repositorio público de GitHub hqc128, mientras que los vectores de prueba oficiales pueden consultarse en el repositorio oficial de HQC. Esto permite la verificación independiente y la reproducibilidad de los resultados obtenidos durante las pruebas del esquema.

4.2.1 Estructura y descripción de funciones

A continuación se presenta un resumen de las funciones implementadas en hqc128.py. La tabla incluye la descripción general de cada función, así como sus parámetros de entrada y salida. Todas las funciones están vinculadas directamente a su ubicación en el código dentro del repositorio HQC-128 en GitHub , lo que permite acceder rápidamente a su implementación detallada.

La tabla provee una descripción para cada función a modo de resumen de la operación que se realiza, a su vez se incluyen los argumentos que la función recibe y los resultados que devuelve, facilitando la comprensión del flujo de datos en el algoritmo. Esta estructuración modular y documentada permite estudiar y verificar cada operación del esquema HQC de manera clara y reproducible.

A continuación se muestra la tabla con las funciones más relevantes:

Tabla 4.1: Resumen de funciones de la implementación en Python del algoritmo HQC

Función	Descripción
__init__()	Esta función inicializa todos los parámetros del esquema HQC en su nivel 128, incluyendo las dimensiones del código, pesos de los vectores dispersos, tamaños de claves y ciphertexts, y constantes de seguridad. Esta etapa define completamente la configuración de seguridad del sistema y garantiza la coherencia con los parámetros especificados en el estándar oficial.
Funciones internas de expansión y muestreo	
xof_init()	Recibe como argumento un arreglo de bytes y realiza la separación de dominio agregando el byte \01. A continuación, inicializa una función de expansión de salida extendible (XOF) basada en SHAKE256, generando flujos pseudoaleatorios reproducibles a partir de la semilla proporcionada.
xof_get_bytes	Extrae una cantidad determinada de bytes desde una instancia XOF previamente inicializada. La función se encarga de que la lectura se realice de forma ordenada y consistente, evitando irregularidades en los valores obtenidos.
sample_fixed_wt_mod()	Genera un vector binario de longitud n y peso fijo wt mediante muestreo modular. Este procedimiento selecciona posiciones pseudoaleatorias y construye un vector disperso sin necesidad de rechazo, se usa durante el proceso de cifrado.

Tabla 4.1 – Continuación

sample_fixed_wt_rej()	Implementa un muestreo por rechazo para generar un vector binario de peso fijo. A diferencia del método modular, este enfoque garantiza una distribución uniforme sobre el conjunto de vectores válidos, siendo empleado durante la generación de claves privadas del PKE.
sample_vect()	Genera un vector binario uniforme de longitud n, truncando los bits excedentes del flujo pseudoaleatorio. Este vector representa un polinomio aleatorio utilizado en la generación de la clave pública.
Operaciones algebraicas	
vect_mul()	Implementa la multiplicación polinomial binaria entre dos vectores, seguida de una reducción módulo $x^n - 1$. Esta operación constituye el núcleo algebraico del algoritmo y modela la convolución circular en el espacio binario.
gf_mul()	Realiza la multiplicación de dos elementos en el campo finito $GF(2^8)$, utilizando un polinomio irreducible estándar. Esta operación es necesaria para la codificación y decodificación Reed–Solomon.
gf_exp()	Calcula la exponenciación de un elemento en $GF(2^8)$ mediante exponenciación rápida. Se emplea como operación auxiliar en la obtención de inversos multiplicativos.
gf_inv()	Calcula el inverso multiplicativo de un elemento en $GF(2^8)$ utilizando la propiedad $x^{-1} = x^{254}$. Esta función es necesaria en los algoritmos de decodificación Reed–Solomon.
gf_gauss()	Aplica eliminación Gaussiana sobre una matriz definida en $GF(2^8)$, permitiendo la resolución de sistemas lineales. Este procedimiento se utiliza durante la corrección de errores en la decodificación Reed–Solomon.
Codificadores de corrección de errores	
rs_genpoly()	Genera el polinomio generador del código Reed–Solomon con capacidad de corrección de t símbolos erróneos. Este polinomio es calculado dinámicamente durante la inicialización del esquema.
rs_encode()	Codifica un mensaje de longitud k mediante Reed–Solomon, produciendo una palabra de código sistemática que incorpora redundancia suficiente para tolerar errores inducidos por el canal criptográfico.

Tabla 4.1 – Continuación

rm_encode()	Codifica el mensaje mediante un código Reed–Muller, expandiendo cada bit en un patrón estructurado que facilita la corrección de errores por decisión mayoritaria en el receptor.
Decodificadores de corrección de errores	
rm_decode()	Decodifica una palabra de código Reed–Muller aplicando una técnica de votación mayoritaria sobre bloques del ciphertext, recuperando el mensaje original incluso en presencia de errores.
rs_synd()	Calcula el síndrome Reed–Solomon de la palabra recibida, el cual resume la información necesaria para localizar y corregir errores durante la decodificación.
rs_elp()	Implementa el algoritmo de Berlekamp–Massey, obteniendo el polinomio localizador de errores a partir del síndrome calculado.
rs_roots()	Determina las posiciones de error evaluando el polinomio localizador sobre los elementos del campo finito, identificando los símbolos erróneos en la palabra recibida.
rs_decode()	Realiza el proceso completo de decodificación Reed–Solomon: cálculo de síndromes, localización de errores, corrección de la palabra y extracción del mensaje original.
Esquema PKE	
pke_keygen()	Genera el par de claves pública y privada del esquema Public Key Encryption, a partir de una semilla determinística. La clave pública incorpora un vector público, mientras que la clave privada conserva la semilla necesaria para reconstruir los vectores secretos.
pke_encrypt()	Cifra un mensaje utilizando la clave pública PKE, generando un ciphertext compuesto por dos componentes, uno asociado al ruido y otro al mensaje codificado. Este proceso introduce errores controlados para garantizar seguridad resistente a la computación cuántica.
pke_decrypt()	Descifra el ciphertext PKE usando la clave privada, aplicando primero operaciones algebraicas y posteriormente decodificación Reed–Muller y Reed–Solomon para recuperar el mensaje original.
Esquema KEM	
keygen()	Genera el par de claves del Key Encapsulation Mechanism, integrando el esquema PKE con material secreto adicional para permitir desacapsulación segura.

Tabla 4.1 – Continuación

kem_encaps()	Realiza el proceso de encapsulación, generando un ciphertext y una clave compartida a partir de la clave pública del receptor y valores aleatorios.
kem_decaps()	Ejecuta la desacapsulación, recuperando la clave compartida y verificando la validez del ciphertext. En caso de fallo, se deriva una clave alternativa para prevenir ataques de canal lateral.
Funciónes de prueba	
prng_init()	Inicializa una instancia SHAKE sobre una seed determinista incremental de 48 bytes concatenada con el byte \00, el cual es su separador de dominio correspondiente.
run_kat_test() test_seed_esp()	Estas funciones implementan pruebas KAT (*Known Answer Tests*), utilizando semillas determinísticas para verificar la correcta conformidad de la implementación con los vectores oficiales del estándar HQC-128.

4.3 Fase 3: Descomposición funcional del algoritmo HQC

En esta etapa se presenta el desglose detallado del algoritmo HQC, describiendo paso a paso las operaciones internas que conforman sus distintos procesos criptográficos. Con el fin de facilitar la comprensión del funcionamiento del algoritmo y asegurar una correcta implementación, se emplean diagramas de flujo de datos y diagramas de bloques, los cuales permiten visualizar el recorrido de la información entre los bloques funcionales y las dependencias existentes entre las operaciones.

Cada diagrama se acompaña de una descripción textual que explica las transformaciones realizadas sobre los datos, así como su correspondencia directa con operaciones en alto nivel, expresadas mediante pseudocódigo o estructuras equivalentes Python. De esta manera, se establece un puente claro entre la especificación teórica del estándar y su implementación práctica, sentando las bases para la posterior traducción del algoritmo a una arquitectura hardware descrita en VHDL.

Esta sección no pretende redefinir el algoritmo, sino interpretar su funcionamiento interno desde una perspectiva de ingeniería, con el objetivo de facilitar su implementación correcta y verificable.

4.3.1 Generación de claves (Keygen)

La generación de claves constituye la etapa inicial del algoritmo HQC y establece los parámetros criptográficos necesarios para los procesos posteriores de encapsulación y desencapsulación. En este apartado se describe el flujo interno del procedimiento de *Key Generation* (KeyGen), detallando las operaciones realizadas, las estructuras de datos involucradas y la forma en que se obtienen las claves pública y privada.

Para facilitar su comprensión, el proceso se analiza de manera secuencial, apoyándose en diagramas de flujo de datos que permiten visualizar las transformaciones aplicadas a la información y su correspondencia con una descripción en alto nivel del algoritmo.

4.3.1.1 Expansión de semillas

Para la validación funcional del algoritmo se emplearon los *Known Answer Tests* (KATs), los cuales permiten verificar la correcta implementación comparando los resultados obtenidos con salidas previamente conocidas.

En este contexto, se utilizó una seed determinista incremental de 48 bytes **inc_seed**, definida como la secuencia hexadecimal consecutiva desde 0x00 hasta 0x2F, la cual constituye la seed oficial utilizada para la generación de los *test vectors* del algoritmo.

A partir de la semilla inicial del PRNG, se derivan 100 semillas independientes para los KATs (count 0 a 99). Cada una de estas semillas de iteración se utiliza para generar la semilla del KEM, que a su vez inicializa el estado SHAKE256 mediante xor_init(), haciendo que cada KAT cuente con un flujo determinístico, el flujo de datos entrega la instancia **xof_state** inicializada.

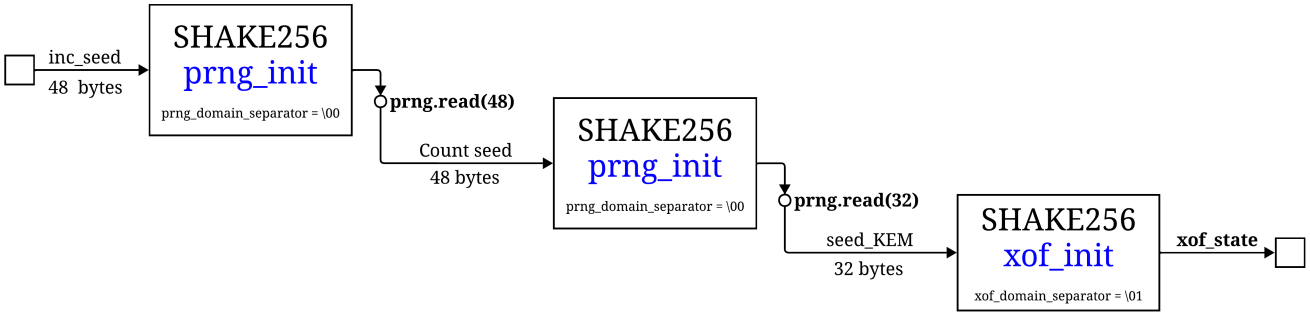


Figura 4.1: Expansión de semillas para HQC-KEM.Keygen()

4.4 CONCLUSIONES

5. PRUEBAS Y RESULTADOS

5.1 INTRODUCCIÓN

5.2 PRUEBA N

5.2.1 Descripción

5.2.2 Procedimiento

5.2.3 Análisis

6. CONCLUSIONES GENERALES

7. TRABAJOS FUTUROS

REFERENCIAS

[1] C. Gidney and M. Ekerå, "How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[2] National Institute of Standards and Technology (NIST), "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," Internal Report (NISTIR) 8545, NIST, March 2025.

[3] IBM, "What is quantum-safe cryptography?." `https://www.ibm.com/think/topics/quantum-safe-cryptography`, September 2024.

[4] S. Singh and E. Sakk, "Implementation and analysis of shor's algorithm to break rsa cryptosystem security," *TechRxiv*, January 2024.

[5] A. T. Olutimehin, S. Joseph, O. C. Metibemu, A. Y. Balogun, O. O. Olaniyi, and A. O. Olayemi, "Future-proofing data: Assessing the feasibility of post-quantum cryptographic algorithms to mitigate 'harvest now, decrypt later' attacks," *Archives of Current Research International*, vol. 25, p. 1098, February 2025.

[6] Cybersecurity and Infrastructure Security Agency (CISA), "Prepare for a new cryptographic standard to protect against future quantum-based threats." CISA, July 2022.

[7] Y. Laigle-Chapuy, O. Blazy, P. Gaborit, A. Hauteville, and J.-P. Tillich, "Hqc: Ind-cca2 kem based on quasi-cyclic codes, specification v1.1," Tech. Rep. NIST PQC Submission Document, National Institute of Standards and Technology (NIST), February 2025.

[8] National Institute of Standards and Technology, "Nist selects hqc as fifth algorithm for post-quantum encryption." NIST News, March 2025.